Team Reference Document

Competitive Programming Templates | source: MuhammadUzzam/Hazrat-CP-Templates

Contents

1. Base Templates . . . 1
   - Base template (general.cpp) . . . 1
   - Extra macros used by Graph Tutorial files . . . 1
   - Custom hash for unordered_map (hash.cpp) . . . 1
   - __int128 input/output (int128.cpp) . . . 2
   - Interactive problems (interactive.cpp) . . . 2
2. Math . . . 2
   - Math template: modpow, inverse, nCr, sieve, matrix exp, extgcd (math_template.txt) . 2
   - Divisors (divisors.cpp) . . . 3
   - Python: nCr + matrix exponentiation (python_template.txt) . . . 3
   - Epsilon binary search on long double (from class screenshot) . . . 4
3. Data Structures . . . 4
   - DSU: array version, path halving + union by size (dsu.cpp) . . . 4
   - DSU: class version (dsu_class.cpp) . . . 4
   - DSU with merge: multiset of component sizes (dsu with merge.cpp) . . . 4
   - DSU: difference array on DSU, join/add/get (difference array on dsu.cpp) . . . 4
   - Segment tree: point update, range sum (SegmentTree.cpp) . . . 5
   - Segment tree: simple array version, range AND (simple_seg_tree.cpp) . . . 5
   - Segment tree: lazy propagation, range add + range sum (LazySegmentTree.cpp) . . 5
   - Segment tree: range add, point query (range_update.cpp) . . . 5
   - Merge sort tree: count elements > k in [l,r] (merge_sort_tree.cpp) . . . 6
   - Segment tree + coordinate compression, offline queries (coordinate_compression.cpp)6
4. Graphs . . . 7
   - Graph representations (representations.cpp) . . . 7
   - Weighted graph representations (weight_representations.cpp) . . . 7
   - BFS levels on a tree (bfs.cpp) . . . 7
   - 0-1 BFS (01_bfs.cpp) . . . 7
   - Multi-source BFS on grid, 8 directions (multi_source_bfs.cpp) . . . 7
   - Knight moves BFS on 8x8 board (nakanj.cpp) . . . 8
   - Connected components with DFS (connected_components.cpp) . . . 8
   - Cycle detection, undirected (cycle.cpp) . . . 8
   - Height and depth of tree nodes (height_depth.cpp) . . . 8
   - Subtree sum / count (subtree_sum.cpp) . . . 8
   - Edge deletion, max product of two parts (edge_deletion.cpp) . . . 9
   - Tree diameter, two DFS (diameter.cpp) . . . 9
   - LCA via root paths (lca.cpp) . . . 9
   - Dijkstra with set (dijkstra.cpp) . . . 9
   - Floyd-Warshall (floyd_warshall.cpp) . . . 9
   - CF Greg and Graph: reverse-order Floyd-Warshall (Greg and Graph_cf.cpp) . . . 9
   - Tarjan SCC + condensation DAG (Tarjan.cpp) . . . 10
   - Kosaraju SCC + condensation DAG (kosaraju.cpp) . . . 10
   - Topological sort, Kahn (kahn's algo (topo sort).cpp) . . . 10
   - Topological sort + cycle check, DFS (topo sort.cpp) . . . 10
5. Dynamic Programming . . . 11
   - Digit DP (digitDP.cpp) . . . 11
   - TSP bitmask DP, recursive (tsp.cpp) . . . 11
   - TSP bitmask DP, iterative (tsp_iterative.cpp) . . . 11
6. Extra . . . 11
   - Codeforces problem filter by tag and rating (cf problem filter, python) . . . 11

1. Base Templates

Base template (general.cpp)

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;
#define hazrat ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);int tt =
  1;
#define int long long int
#define vi vector<int>
#define vb vector<bool>
#define vc vector<char>
#define vs vector<string>
#define vvi vector<vi>
#define iv(vec) for(auto &value: vec) cin>>value;
#define pr(vec) {for(auto value: vec) cout<<value<<" ";} nl;
#define pb push_back
#define all(a) a.begin(), a.end()
#define rall(a) a.rbegin(), a.rend()
#define pii pair<int,int>
#define vpii vector<pii>
#define ld long double
#define nl cout<<"\n";
#define yes cout << "YES\n"
#define no cout << "NO\n"
#define file_read(filepath) freopen(filepath, "r", stdin);
#define file_write(filepath) freopen(filepath, "w", stdout);
const int mod=1e9+7;
const int mod2=998244353;
const int INF=4e18;
void solve(){
}
signed main(){
  hazrat;
  cin>>tt;
  while(tt--){
    solve();
  }
  return 0;
}
```

Extra macros used by Graph Tutorial files

Headers (#include / common #define lines) are stripped from the files below; they assume the base template above.

```
#define fast ios::sync_with_stdio(false); cin.tie(0);
#define rep(i,a,b) for(int i=a; i<b; i++)
#define per(i,a,b) for(int i=b-1; i>=a; i--)
```

Custom hash for unordered_map (hash.cpp)

```
const int MOD = 1e9+7;
const int INF = 1e9;
// ================ CUSTOM HASH =================
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + RANDOM);
    }
    size_t operator()(const pii &p) const {
        static const uint64_t RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(p.first + RANDOM) ^
               (splitmix64(p.second + RANDOM) >> 1);
    }
    size_t operator()(const vi &v) const {
        uint64_t hash = v.size();
        for (auto x : v) {
            hash ^= splitmix64(x + 0x9e3779b97f4a7c15 +
                    (hash << 6) + (hash >> 2));
        }
        return hash;
    }
};
// ================ SOLVE =================
void solve() {
    // int key
    unordered_map<int,int,custom_hash> mp1;
    mp1[5]++;
    cout << mp1[5]; nl;
    // pair key
    unordered_map<pii,int,custom_hash> mp2;
    mp2[{1,2}] = 10;
    cout << mp2[{1,2}]; nl;
    // vector key
    unordered_map<vi,int,custom_hash> mp3;
    vi v = {1,2,3};
    mp3[v] = 100;
    cout << mp3[v]; nl;
}
```

```
// ================= MAIN =================
signed main() {
    hazrat;
    cin >> tt;
    while (tt--) {
        solve();
    }
    return 0;
}
```

__int128 input/output (int128.cpp)

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;
// Define shorter aliases for convenience
using int128 = __int128;
using uint128 = unsigned __int128;
// Overload cin >> for __int128
istream& operator>>(istream& is, int128& v) {
  string s;
  if (!(is >> s)) return is;
  v = 0;
  bool neg = false;
  int i = 0;
  if (s[i] == '-') { neg = true; i++; }
  else if (s[i] == '+') { i++; }
  for (; i < s.length(); i++) {
    if (s[i] >= '0' && s[i] <= '9') {
      v = v * 10 + (s[i] - '0');
    }
  }
  if (neg) v = -v;
  return is;
}
// Overload cout << for __int128
ostream& operator<<(ostream& os, int128 v) {
  if (v == 0) return os << "0";
  if (v < 0) { os << "-"; v = -v; }
  string s;
  while (v > 0) {
    s += (char)('0' + (v % 10));
    v /= 10;
  }
  reverse(s.begin(), s.end());
  return os << s;
}
int main() {
  // Optimize standard I/O operations for speed
  ios_base::sync_with_stdio(false);
  cin.tie(NULL);
  int128 a, b;
  // Read __int128 directly using cin
  if (cin >> a >> b) {
    int128 result = a * b;
    // Print __int128 directly using cout
    cout << result << "\n";
  }
  return 0;
}
```

Interactive problems (interactive.cpp)

```
// ya to cout.flush() use kro ya phr "endl" krlo use.
int query(int lo,int hi){
  cout<<"? "<<hi-lo+1<<" ";
  for(int i=lo;i<=hi;++i){
    cout<<i+1<<" \n"[i==hi];
  }
  cout.flush();
  int res;cin>>res;
  return res;
}
void solve(){
  int n;cin>>n;
  vi a(n);iv(a);
  vi pa(n+1);
  for(int i=1;i<=n;++i){
    pa[i]=pa[i-1]+a[i-1];
  }
  int lo=0,hi=n-1;
  while(lo<=hi){
    int mid=lo+(hi-lo)/2;
    if(lo==hi){
      cout<<"! "<<lo+1<<"\n";cout.flush();return;
    }
    if(query(lo,mid)==pa[mid+1]-pa[lo]){
      lo=mid+1;
    }else{
      hi=mid;
    }
  }
}
```

2. Math

Math template: modpow, inverse, nCr, sieve, matrix exp, extgcd (math_template.txt)

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define vi vector<int>
#define vvi vector<vector<int>>
// ======================
// Constants
// ======================
const int mod=1e9+7;
const int MAXN=2000001; // for nCr (2*n safe)
// ======================
// modular helpers
// ======================
// normalize in [0, mod-1]
int normalize(int x){
  x%=mod;
  if(x<0) x+=mod;
  return x;
}
// binary exponentiation
int binexp(int a,int b){
  a=normalize(a);
  int res=1;
  while(b){
    if(b&1) res=res*a%mod;
    a=a*a%mod;
    b>>=1;
  }
  return res;
}
// binexp computes (a^b) % mod for any a and b>=0 (negative b loops
  forever)
// If mod is prime and gcd(a, mod) = 1, then modular inverse of a is:
  ((a%mod)^(mod-2)) % mod
// modular inverse (mod must be prime)
int inv(int a){
  return binexp(a,mod-2);
}
// ======================
// GCD / LCM
// ======================
// gcd(x,y) = gcd(x-y,y)
// If x.y is divisible by a.b then y must be divisible by a.b/gcd(a.b,x)
int gcdll(int a,int b){
  return gcd(a,b); // std::gcd (C++17), always non-negative
}
int lcmll(int a,int b){
  return a/gcdll(a,b)*b;
}
// ======================
// Extract bits from numbers
// ======================
string bits(int it){
  bitset<64> bits(it);
  string ans=bits.to_string();
  return ans;
}
// a ^ b = a + b - 2 * (a & b)
// result of a ^ b and a + b have same parity
// ======================
// Factorials + nCr
// ======================
int fact[MAXN],invfact[MAXN];
void init_nCr(){
  fact[0]=1;
  for(int i=1;i<MAXN;i++)
    fact[i]=fact[i-1]*i%mod;
  invfact[MAXN-1]=inv(fact[MAXN-1]);
  for(int i=MAXN-2;i>=0;i--)
    invfact[i]=invfact[i+1]*(i+1)%mod;
}
int nCr(int n,int r){
  if(r<0 || r>n) return 0;
  return fact[n]*invfact[r]%mod*invfact[n-r]%mod;
}
// 'n' stars and 'k-1' bars
// x1 + x2 + ... + xk = n, xi >= 0
int stars_bars(int n,int k){
  return nCr(n+k-1,k-1);
  // or nCr(n+k-1,n)
}
// x1 + x2 + ... + xk = n, xi >= 1
int stars_bars_positive(int n,int k){
  return nCr(n-1,k-1);
}
// xi >= 0  → C(n+k-1, k-1)
// xi >= 1  → C(n-1, k-1) → C((n-k)+k-1, k-1)
// xi >= L  → C((n-L*k)+k-1, k-1)
// if 'L' is different for every 'xi' then replace L*k of above equation
// with sum of 'L' of every 'xi'
// ======================
// nPr (permutations)
// ======================
int nPr(int n,int r){
  if(r<0 || r>n) return 0;
  return fact[n]*invfact[n-r]%mod;
}
// ======================
// Prefix factorial inverse (rare but useful)
// ======================
// inv[i] = modular inverse of i
int invNum[MAXN];
void init_inv(){
  invNum[1]=1;
  for(int i=2;i<MAXN;i++)
    invNum[i]=mod-(mod/i)*invNum[mod%i]%mod;
}
// ======================
// Sieve (primes)
// ======================
const int SIEVE_N=1e6+5;
vector<int> primes;
vector<bool> isPrime(SIEVE_N,true);
void sieve(){
  isPrime[0]=isPrime[1]=false;
  for(int i=2;i*i<SIEVE_N;i++){
    if(isPrime[i]){
      for(int j=i*i;j<SIEVE_N;j+=i)
        isPrime[j]=false;
    }
  }
  for(int i=2;i<SIEVE_N;i++)
    if(isPrime[i])
      primes.push_back(i);
}
// ======================
// Linear Sieve (SPF)
// ======================
// spf[x] = smallest prime factor of x, built in O(N)
const int SPF_N=1e6+5;
int spf[SPF_N];
vector<int> lp_primes;
```

```
void linear_sieve(){
  for(int i=2;i<SPF_N;i++){
    if(spf[i]==0){
      spf[i]=i;
      lp_primes.push_back(i);
    }
    for(int p:lp_primes){
      if(p>spf[i] || i*p>=SPF_N) break;
      spf[i*p]=p;
    }
  }
}
// prime factorization using spf in O(log x), x < SPF_N
vector<int> factorize_spf(int x){
  vector<int> f;
  while(x>1){
    f.push_back(spf[x]);
    x/=spf[x];
  }
  return f;
}
// =====================
// Fast prime check
// =====================
bool is_prime(int n){
  if(n<=1) return false;
  if(n<=3) return true;
  if(n%2==0 || n%3==0) return false;
  for(int i=5;i*i<=n;i+=6)
    if(n%i==0 || n%(i+2)==0)
      return false;
  return true;
}
// =====================
// Prime factorization
// =====================
vector<int> factorize(int n){
  vector<int> f;
  for(int i=2;i*i<=n;i++){
    while(n%i==0){
      f.push_back(i);
      n/=i;
    }
  }
  if(n>1) f.push_back(n);
  return f;
}
// =====================
// MEX
// =====================
int mex(vector<int>& a){
  int n=a.size();
  vector<bool> seen(n+1,false); // mex of n numbers is always <= n
  for(int x:a) if(x>=0 && x<=n) seen[x]=true;
  int m=0;
  while(seen[m]) m++;
  return m;
}
// =====================
// Sum of digits
// =====================
int sum_digits(int n){
  int s=0;
  while(n){
    s+=n%10;
    n/=10;
  }
  return s;
}
// =====================
// Matrix Exponentiation
// =====================
struct Matrix{
  int n,m;
  vector<vector<int>> a;
  Matrix(int n,int m):n(n),m(m),a(n,vector<int>(m,0)){}
  static Matrix identity(int n){
    Matrix I(n,n);
    for(int i=0;i<n;i++) I.a[i][i]=1;
    return I;
  }
};
Matrix multiply(const Matrix& A,const Matrix& B){
  Matrix C(A.n,B.m);
  for(int i=0;i<A.n;i++){
    for(int k=0;k<A.m;k++){
      if(A.a[i][k]==0) continue;
      for(int j=0;j<B.m;j++)
        C.a[i][j]=(C.a[i][j]+A.a[i][k]*B.a[k][j])%mod;
    }
  }
  return C;
}
Matrix matpow(Matrix A,int p){
  Matrix res=Matrix::identity(A.n);
  while(p){
    if(p&1) res=multiply(res,A);
    A=multiply(A,A);
    p>>=1;
  }
  return res;
}
// =====================
// Extended GCD
// =====================
int extgcd(int a,int b,int& x,int& y){
  if(b==0){
    x=1;
    y=0;
    return a;
  }
  int x1,y1;
  int g=extgcd(b,a%b,x1,y1);
  x=y1;
  y=x1-(a/b)*y1;
  return g;
}
/* Divisibility Rules (CP Quick Check)
2 → last_digit % 2 == 0
3 → (sum_of_digits % 3) == 0
4 → (last_two_digits % 4) == 0
5 → last_digit % 5 == 0 // (0 or 5)
6 → (n % 2 == 0) && (sum_of_digits % 3 == 0)
8 → (last_three_digits % 8) == 0
9 → (sum_of_digits % 9) == 0
10 → last_digit % 10 == 0 // last digit = 0
11 → ((sum_odd_positions - sum_even_positions) % 11) == 0
25 → (last_two_digits % 25) == 0 // possible endings: 00, 25, 50, 75 */
// =====================
// Generate Subsets
// =====================
vvi generateSubsets(vi& a){
  int n=a.size();
  vvi subsets;
  for(int i=0;i<(1<<n);++i){
    vi curr;
    for(int j=0;j<n;++j){
      if(i&(1<<j))
        curr.push_back(a[j]);
    }
    subsets.push_back(curr);
  }
  return subsets;
}
// Calculating array size constraints.
/*
Let's say we have to use a vector of 50000 * 500 size
so we calculate it like that:
First multiply 50000 * 500 = 25000000 then multiply it
with data type of vector like 8 for long long, 4 for int etc. Then
  divide answer by 1000000 to get the size in MBs
the array will take. A sample is shown below:
25000000 * 8 = 200000000
200000000 / 1000000 = 200 MB
the array will take.
*/
// Perfect Squares have odd number of divisors.
void solve(){
}
signed main(){
  ios::sync_with_stdio(false);cin.tie(nullptr);
  int t=1;cin>>t;
  while(t--) solve();
}
```

Divisors (divisors.cpp)

```
// Divisors
// Number of divisors
int cntDiv(int n){
  int cnt=0;
  for(int i=1;i*i<=n;++i){
    if(n%i==0){
      cnt++;
      if(i!=n/i) cnt++;
    }
  }
  return cnt;
}
// Sum of divisors
int sumDiv(int n){
  int sum=0;
  for(int i=1;i*i<=n;++i){
    if(n%i==0){
      sum+=i;
      if(i!=n/i) sum+=n/i;
    }
  }
  return sum;
}
// Product of divisors
// product = n^(d(n)/2)
// handle perfect square separately if using modulo
// Get all divisors
vector<int> getDiv(int n){
  vector<int>v;
  for(int i=1;i*i<=n;++i){
    if(n%i==0){
      v.push_back(i);
      if(i!=n/i) v.push_back(n/i);
    }
  }
  return v;
}
// Get sorted divisors
vector<int> getDivSorted(int n){
  vector<int>v=getDiv(n);
  sort(v.begin(),v.end());
  return v;
}
// Precompute divisors for 1...N
vector<int>divs[N+1];
for(int i=1;i<=N;++i)
  for(int j=i;j<=N;j+=i)
    divs[j].push_back(i);
// Precompute number of divisors
vector<int>cnt(N+1);
for(int i=1;i<=N;++i)
  for(int j=i;j<=N;j+=i)
    cnt[j]++;
Important Formulas
n = p1^a1 * p2^a2 * ...
Number of divisors:
d(n) = (a1+1)(a2+1)...
Sum of divisors:
σ(n) = Π (1+p+p²+...+p^a)
Product of divisors:
Π divisors = n^(d(n)/2)
```

Python: nCr + matrix exponentiation (python_template.txt)

```
mod=10**9+7
```

```
fact=[1]*(1000000)
for i in range(1,1000000):
  fact[i]=fact[i-1]*i%mod
def nCr(n,r):
  if r<0 or r>n:return 0
  return fact[n]*pow(fact[r]*fact[n-r]%mod,mod-2,mod)%mod
# Matrix Exponentiation
MOD=10**9+7
def multiply(A,B):
  n=len(A)
  C=[[0]*n for _ in range(n)]
  for i in range(n):
    for k in range(n):
      if A[i][k]==0: continue
      for j in range(n):
        C[i][j]=(C[i][j]+A[i][k]*B[k][j])%MOD
  return C
def matpow(A,p):
  n=len(A)
  res=[[1 if i==j else 0 for j in range(n)] for i in range(n)]
  while p:
    if p&1:
      res=multiply(res,A)
    A=multiply(A,A)
    p>>=1
```

Epsilon binary search on long double (from class screenshot)

```
ld lo = 2, hi = d;
while (hi - lo > 1e-7) {
  ld mid = (lo + hi) / 2;
  if ((mid * mid) / (mid - 1) > d) hi = mid;
  else lo = mid;
}
ld a = lo;
cout << a << " " << d - a << endl;
```

3. Data Structures

DSU: array version, path halving + union by size (dsu.cpp)

```
const int N=5e5+10;
int parent[N];  // ye immediate parent of a node store krega
int sz[N];
void make(int v){
  parent[v]=v;
  sz[v]=1;
}
// ye group ka leader return krega.
int find(int v){
  while(v!=parent[v]){
    parent[v]=parent[parent[v]];
    v=parent[v];
  }
  return v;
}
void Union(int a,int b){
  a=find(a);
  b=find(b);
  if(a!=b){
    // Union by size
    if(sz[a]<sz[b]) swap(a,b);
    parent[b]=a;
    sz[a]+=sz[b];
  }
}
// Sample
void solve(){
  int n,m;cin>>n>>m;
  // distinct connected components
  for(int i=1;i<=n;++i){
    make(i);
  }
  while(m--){
    int a,b;cin>>a>>b;
    Union(a,b);
  }
  int cc=0;
  for(int i=1;i<=n;++i){
    if(find(i)==i){
      cc++;
    }
  }
  cout<<cc<<endl;
}
```

DSU: class version (dsu_class.cpp)

```
class DSU{
public:
  vector<int>par,sz;
  DSU(int n){
    par.resize(n+1);
    sz.assign(n+1,1);
    iota(par.begin(),par.end(),1);
  }
  int find(int x){
    return par[x]==x?x:par[x]=find(par[x]);
  }
  void unite(int a,int b){
    a=find(a),b=find(b);
    if(a==b)return;
    if(sz[a]<sz[b])swap(a,b);
    par[b]=a;
    sz[a]+=sz[b];
  }
  bool same(int a,int b){
    return find(a)==find(b);
  }
  int size(int x){
    return sz[find(x)];
  }
};
```

DSU with merge: multiset of component sizes (dsu with merge.cpp)

```
#define f(i, a, n) for (int i = a; i < n; i++)
#define uset unordered_set
#define mset unordered_multiset
#define ned ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);int tt = 1;
const int N=2e5+10;
int parent[N];
vector<int>sz(N);
multiset<int>res;
void make(int v){
  parent[v]=v;
  sz[v]=1;
  res.insert(1);
}
int find(int v){
  if(v==parent[v])    return v;
  // Path Compression
  return parent[v]=find(parent[v]);
}
void merge(int a,int b){
  res.erase(res.find(sz[a]));
  res.erase(res.find(sz[b]));
  res.insert(sz[a]+sz[b]);
}
void Union(int a,int b){
  a=find(a);
  b=find(b);
  if(a!=b){
    // Union by size
    if(sz[a]<sz[b]) swap(a,b);
    parent[b]=a;
    merge(a,b);
    sz[a]+=sz[b];
  }
}
void solve(){
  int n,q;cin>>n>>q;
  for(int i=1;i<=n;++i){
    make(i);
  }
  while(q--){
    int a,b;cin>>a>>b;
    Union(a,b);
    if(res.size()==1){
      cout<<0<<endl;
    }else{
      cout<<(*prev(res.end())-*res.begin())<<endl;
    }
  }
}
signed main(){
  ned;
  // cin>>tt;
  while(tt--)
  {
    solve();
  }
  return 0;
}
```

DSU: difference array on DSU, join/add/get (difference array on dsu.cpp)

```
const int N=5e5+10;
int parent[N];  // ye immediate parent of a node store krega
int sz[N];
int res[N];
void make(int v){
  parent[v]=v;
  sz[v]=1;
}
// ye group ka leader return krega.
int find(int v){
  // ismein complexity amortized nhi hogi but union by size ki wja se
    log(n) rhe gi so its acceptable.
  /*
  amortized na rkhne ka reason ye hai k baaz dafa humein parent ko har
    find ki query mein update
  nhi krna hota.
  */
  while(v!=parent[v]){
    v=parent[v];
  }
  return v;
}
void Union(int a,int b){
  a=find(a);
  b=find(b);
  if(a!=b){
    // Union by size
    if(sz[a]<sz[b]) swap(a,b);
    parent[b]=a;
    sz[a]+=sz[b];
    res[b]-=res[a];
  }
}
int get(int v){
  int curr=res[v];
  while(v!=parent[v]){
    curr+=res[parent[v]];
    v=parent[v];
  }
  return curr;
}
// Sample
void solve(){
  int n,m;cin>>n>>m;
  // distinct connected components
  for(int i=1;i<=n;++i){
    make(i);
  }
  while(m--){
    string s;cin>>s;
    if(s=="join"){
      int a,b;cin>>a>>b;
      Union(a,b);
    }else if(s=="add"){
      int u,v;cin>>u>>v;
      res[find(u)]+=v;
    }else{
```

```
      int u;cin>>u;
      cout<<get(u)<<"\n";
    }
  }
}
/*
is implementation se har add operation mein jo node hogi uske poora
  group mein relevant addition hojayegi
and get se hum classic difference array technique se wo addition
  finalized krlein gai.
*/
```

Segment tree: point update, range sum (SegmentTree.cpp)

```
struct node{
  int curr;
  node(){
    curr=0;
  }
  node(int curr){
    this->curr=curr;
  }
};
class SegmentTree{
  public:
    vector<int>arr;
    vector<node>tree;
    int n;
    void build(int idx,int l,int r){
      if(l==r){
        tree[idx].curr=arr[l];
        return;
      }
      int mid=l+(r-l)/2;
      build(2*idx+1,l,mid);
      build(2*idx+2,mid+1,r);
      tree[idx]=merge(tree[2*idx+1],tree[2*idx+2]);
    }
    node merge(node a,node b){
      return node(a.curr+b.curr);
    }
    node query(int idx,int l,int r,int lq,int rq){
      if(rq<l||lq>r)  return node();
      if(lq<=l&&rq>=r)    return tree[idx];
      int mid=l+(r-l)/2;
      node res=merge(query(2*idx+1,l,mid,lq,rq),query(2*idx+2,mid+1,r,lq
        ,rq));
      return res;
    }
    void update(int idx,int l,int r,int p,int v){
      if(l==r){
        arr[l]=v;
        tree[idx].curr=v;
        return;
      }
      int mid=l+(r-l)/2;
      if(p<=mid)  update(2*idx+1,l,mid,p,v);
      else update(2*idx+2,mid+1,r,p,v);
      tree[idx]=merge(tree[2*idx+1],tree[2*idx+2]);
    }
    SegmentTree(vector<int>&vec){
      arr=vec;
      n=arr.size();
      tree.resize(4*n);
      build(0,0,n-1);
    }
    node getSum(int l,int r){
      return query(0,0,n-1,l,r);
    }
    void setValue(int p,int v){
      update(0,0,n-1,p,v);
    }
};
signed main(){
  int n,q;cin>>n>>q;
  vector<int>a(n);
  for(auto &it:a) cin>>it;
  SegmentTree seg(a);
  while(q--){
    int t;cin>>t;
    if(t==1){
      int i,v;cin>>i>>v;
      seg.setValue(i,v);
    }else{
      int l,r;cin>>l>>r;
      cout<<seg.getSum(l,r).curr<<"\n";
    }
  }
  return 0;
}
```

Segment tree: simple array version, range AND (simple_seg_tree.cpp)

```
struct node {
  int sum;
  node() {
    sum = 0;
  }
};
node seg[4 * 200200];
int a[200200];
node merge(node left, node right) {
  node ans;
  ans.sum = (left.sum & right.sum);
  return ans;
}
void build(int ind, int low, int high) {
  if (low == high) {
    seg[ind].sum = a[low];
    return;
  }
  int mid = (low + high) >> 1;
  build(2 * ind, low, mid);
  build(2 * ind + 1, mid + 1, high);
  seg[ind] = merge(seg[2 * ind], seg[2 * ind + 1]);
}
node query(int ind, int low, int high, int lq, int rq) {
  if (rq < low || high < lq) {
    node ans;
    ans.sum=LLONG_MAX;  // this is use when you want to find bitwise
      AND of intervals
    return ans;
  }
  if (lq <= low && high <= rq) {
    return seg[ind];
  }
  int mid = (low + high) >> 1;
  return merge(
    query(2 * ind, low, mid, lq, rq),
    query(2 * ind + 1, mid + 1, high, lq, rq)
  );
}
```

Segment tree: lazy propagation, range add + range sum (LazySegmentTree.cpp)

```
struct node{
  int curr;
  node(){
    curr=0;
  }
  node(int curr){
    this->curr=curr;
  }
};
class LazySegmentTree{
  public:
    vector<int>arr;
    vector<node>segTree;
    vector<node>lazy;
    int n;
    void build(int idx,int l,int r){
      if(l==r){
        segTree[idx].curr=arr[l];
        return;
      }
      int mid=l+(r-l)/2;
      build(2*idx+1,l,mid);
      build(2*idx+2,mid+1,r);
      segTree[idx]=merge(segTree[2*idx+1],segTree[2*idx+2]);
    }
    node merge(node a,node b){
      return node(a.curr+b.curr);
    }
    void push(int idx,int l,int r){
      if(lazy[idx].curr!=0){
        segTree[idx].curr+=(r-l+1)*lazy[idx].curr;
        if(l!=r){
          lazy[2*idx+1].curr+=lazy[idx].curr;
          lazy[2*idx+2].curr+=lazy[idx].curr;
        }
        lazy[idx].curr=0;
      }
    }
    node query(int idx,int l,int r,int lq,int rq){
      push(idx,l,r);
      if(rq<l||lq>r)  return node();
      if(lq<=l&&rq>=r)    return segTree[idx];
      int mid=l+(r-l)/2;
      node res=merge(query(2*idx+1,l,mid,lq,rq),query(2*idx+2,mid+1,r,lq
        ,rq));
      return res;
    }
    void update(int idx,int l,int r,int lq,int rq,int v){
      push(idx,l,r);
      if(rq<l||lq>r)  return;
      if(lq<=l&&rq>=r){
        lazy[idx].curr+=v;
        push(idx,l,r);
        return;
      }
      int mid=l+(r-l)/2;
      update(2*idx+1,l,mid,lq,rq,v);
      update(2*idx+2,mid+1,r,lq,rq,v);
      segTree[idx]=merge(segTree[2*idx+1],segTree[2*idx+2]);
    }
    LazySegmentTree(vector<int>&vec){
      arr=vec;
      n=arr.size();
      segTree.resize(4*n);
      lazy.resize(4*n);
      build(0,0,n-1);
    }
    node getSum(int l,int r){
      return query(0,0,n-1,l,r);
    }
    void setValue(int l,int r,int v){
      update(0,0,n-1,l,r,v);
    }
};
signed main(){
  int n,q;cin>>n>>q;
  vector<int>a(n);
  for(auto &it:a) cin>>it;
  LazySegmentTree seg(a);
  while(q--){
    int t;cin>>t;
    if(t==1){
      int l,r,v;cin>>l>>r>>v;
      seg.setValue(l,r,v);
    }else{
      int l,r;cin>>l>>r;
      cout<<seg.getSum(l,r).curr<<"\n";
    }
  }
  return 0;
}
```

Segment tree: range add, point query (range_update.cpp)

```
struct node{
  int curr;
  node(){
    curr=0;
```

```
  }
  node(int curr){
    this->curr=curr;
  }
};
class SegmentTree{
  public:
    vector<int>arr;
    vector<node>tree;
    int n;
    void build(int idx,int l,int r){
      if(l==r){
        tree[idx].curr=arr[l];
        return;
      }
      int mid=l+(r-l)/2;
      build(2*idx+1,l,mid);
      build(2*idx+2,mid+1,r);
      tree[idx]=merge(tree[2*idx+1],tree[2*idx+2]);
    }
    node merge(node a,node b){
      return node(a.curr+b.curr);
    }
    int query(int idx,int l,int r,int lq,int rq){
      int curr=0;
      if(rq<l||lq>r)  return curr;
      curr+=tree[idx].curr;
      if(l==r){
        return curr;
      }
      int mid=l+(r-l)/2;
      curr+=query(2*idx+1,l,mid,lq,rq);
      curr+=query(2*idx+2,mid+1,r,lq,rq);
      return curr;
    }
    void update(int idx,int l,int r,int lq,int rq,int v){
      if(rq<l||lq>r)  return;
      if(lq<=l&&rq>=r){
        tree[idx].curr+=v;
        return;
      }
      int mid=l+(r-l)/2;
      update(2*idx+1,l,mid,lq,rq,v);
      update(2*idx+2,mid+1,r,lq,rq,v);
    }
    SegmentTree(vector<int>&vec){
      arr=vec;
      n=arr.size();
      tree.resize(4*n);
      build(0,0,n-1);
    }
    int getSum(int l,int r){
      return query(0,0,n-1,l,r);
    }
    void setValue(int l,int r,int v){
      update(0,0,n-1,l,r,v);
    }
};
void solve(){
  int n,m;cin>>n>>m;
  vector<int> a(n,0);
  SegmentTree seg(a);
  while(m--){
    int t;cin>>t;
    if(t==1){
      int l,r,v;cin>>l>>r>>v;
      seg.setValue(l,r-1,v);
    }else{
      int v;cin>>v;
      cout<<seg.getSum(v,v)<<"\n";
    }
  }
}
signed main(){
  int tt=1;
  // cin>>tt;
  while(tt--){
    solve();
  }
  return 0;
}
```

Merge sort tree: count elements > k in [l,r] (merge_sort_tree.cpp)

```
const int MOD=1e9+7;
const int INF=1e18;
struct node{
  vi curr;
};
class SegmentTree{
  public:
    vector<int>arr;
    vector<node>tree;
    int n;
    void build(int idx,int l,int r){
      if(l==r){
        tree[idx].curr.pb(arr[l]);
        return;
      }
      int mid=l+(r-l)/2;
      build(2*idx+1,l,mid);
      build(2*idx+2,mid+1,r);
      tree[idx]=merge(tree[2*idx+1],tree[2*idx+2]);
    }
    node merge(node a,node b){
      node res;
      int i=0,j=0;
      while(i<a.curr.size()&&j<b.curr.size()){
        if(a.curr[i]<b.curr[j]){
          res.curr.pb(a.curr[i++]);
        }else{
          res.curr.pb(b.curr[j++]);
        }
      }
      while(i<a.curr.size()){
        res.curr.pb(a.curr[i++]);
      }
      while(j<b.curr.size()){
        res.curr.pb(b.curr[j++]);
      }
      return res;
    }
    int func(vi&a,int k){
      int res=upper_bound(all(a),k)-a.begin();
      return a.size()-res;
    }
    int query(int idx,int l,int r,int lq,int rq,int k){
      if(rq<l||lq>r)  return 0;
      if(lq<=l&&rq>=r){
        return func(tree[idx].curr,k);
      }
      int mid=l+(r-l)/2;
      int res=query(2*idx+1,l,mid,lq,rq,k)+query(2*idx+2,mid+1,r,lq,rq,k
        );
      return res;
    }
    SegmentTree(vector<int>&vec){
      arr=vec;
      n=arr.size();
      tree.resize(4*n);
      build(0,0,n-1);
    }
    int get(int l,int r,int k){
      return query(0,0,n-1,l,r,k);
    }
};
void solve(){
  int n;cin>>n;
  vi a(n);iv(a);
  SegmentTree seg(a);
  int q;cin>>q;
  while(q--){
    int i,j,k;cin>>i>>j>>k;
    i--;j--;
    cout<<seg.get(i,j,k)<<"\n";
  }
}
signed main(){
  hazrat;
  // cin>>tt;
  while(tt--){
    solve();
  }
  return 0;
}
```

Segment tree + coordinate compression, offline queries (coordinate_compression.cpp)

```
const int MOD=1e9+7;
const int INF=1e18;
struct node{
  int curr;
  node(){
    curr=0;
  }
  node(int curr){
    this->curr=curr;
  }
};
class SegmentTree{
  public:
    vector<int>arr;
    vector<node>tree;
    int n;
    void build(int idx,int l,int r){
      if(l==r){
        tree[idx].curr=arr[l];
        return;
      }
      int mid=l+(r-l)/2;
      build(2*idx+1,l,mid);
      build(2*idx+2,mid+1,r);
      tree[idx]=merge(tree[2*idx+1],tree[2*idx+2]);
    }
    node merge(node a,node b){
      return node(a.curr+b.curr);
    }
    node query(int idx,int l,int r,int lq,int rq){
      if(rq<l||lq>r)  return node();
      if(lq<=l&&rq>=r)    return tree[idx];
      int mid=l+(r-l)/2;
      node res=merge(query(2*idx+1,l,mid,lq,rq),query(2*idx+2,mid+1,r,lq
        ,rq));
      return res;
    }
    void update(int idx,int l,int r,int p,int v){
      if(l==r){
        arr[l]+=v;
        tree[idx].curr+=v;
        return;
      }
      int mid=l+(r-l)/2;
      if(p<=mid)  update(2*idx+1,l,mid,p,v);
      else update(2*idx+2,mid+1,r,p,v);
      tree[idx]=merge(tree[2*idx+1],tree[2*idx+2]);
    }
    SegmentTree(vector<int>&vec){
      arr=vec;
      n=arr.size();
      tree.resize(4*n);
      build(0,0,n-1);
    }
    node getSum(int l,int r){
      return query(0,0,n-1,l,r);
    }
    void setValue(int p,int v){
      update(0,0,n-1,p,v);
    }
};
void solve(){
  int n,q;cin>>n>>q;
```

```
  vi a(n);
  set<int>st;
  for(int i=0;i<n;++i){
    cin>>a[i];
    st.insert(a[i]);
  }
  vector<array<int,3>>opr(q);
  for(int i=0;i<q;++i){
    char x;int y,z;cin>>x>>y>>z;
    opr[i]={x,y,z};
    if(opr[i][0]=='!')  st.insert(opr[i][2]);
    else{
      st.insert(opr[i][1]);
      st.insert(opr[i][2]);
    }
  }
  vi mp(st.size());
  int i=0;
  for(auto it:st){
    mp[i++]=it;
  }
  vi vec(mp.size(),0);
  SegmentTree seg(vec);
  for(int i=0;i<n;++i){
    seg.setValue(lower_bound(all(mp),a[i])-mp.begin(),1);
  }
  for(int i=0;i<q;++i){
    if(opr[i][0]=='!'){
      auto [k,x]=tuple(opr[i][1],opr[i][2]);
      int curr=lower_bound(all(mp),a[k-1])-mp.begin();
      a[k-1]=x;
      seg.setValue(curr,-1);
      seg.setValue(lower_bound(all(mp),x)-mp.begin(),1);
    }else{
      auto [a,b]=tuple(opr[i][1],opr[i][2]);
      cout<<seg.getSum(lower_bound(all(mp),a)-mp.begin(),lower_bound(all
        (mp),b)-mp.begin()).curr<<"\n";
    }
  }
}
signed main(){
  hazrat;
  // cin>>tt;
  while(tt--){
    solve();
  }
  return 0;
}
```

4. Graphs

Graph representations (representations.cpp)

```
const int N = 1e3+10; // Maximum vertices in constraints
int graph1[N][N];
vector<int> graph2[N];
int32_t main(){
  int n, m;
  cin >> n >> m;
  for(int i=0 ; i<m ; ++i){
    int v1, v2;
    cin >> v1 >> v2;
    graph1[v1][v2] = 1;
    graph1[v2][v1] = 1;
    graph2[v1].push_back(v2);
    graph2[v2].push_back(v1);
  }
  // Adjacency Matrix:
  // Space Complexity (N*N)
  // Don't represent graph of N = 1e5 or more (because max space of
    array
  // is 1e7 which is below 1e5*1e5 or even 1e4*1e4)
  // Adjacency List
  // O(N + M) ->N vertices, M edges
  // N = 10 ^ 5 but  edges must be less than 10 ^ 7
  return 0;
}
```

Weighted graph representations (weight_representations.cpp)

```
#define read(v, n) vector<int> v(n); rep(i,0,n) cin >> v[i]
const int N = 1e3+10; // Maximum vertices in constraints
vector<int> graph1[N][N];
vector<pair<int, int>> graph2[N];
int32_t main(){
  int n, m;
  cin >> n >> m;
  for(int i=0 ; i<m ; ++i){
    int v1, v2, wt;
    cin >> v1 >> v2 >> wt;
    graph1[v1][v2] = wt;
    graph1[v2][v1] = wt;
    graph2[v1].push_back({v2, wt});
    graph2[v2].push_back({v1, wt});
  }
  // Adjacency Matrix:
  // Space Complexity (N*N)
  // Don't represent graph of N = 1e5 or more (because max space of
    array
  // is 1e7 which is below 1e5*1e5 or even 1e4*1e4)
  // Adjacency List
  // O(N + M) ->N vertices, M edges
  // N = 10 ^ 5 but  edges must be less than 10 ^ 7
  return 0;
}
```

BFS levels on a tree (bfs.cpp)

```
const int N = 1e5+10;
vector<int> gp[N];
int vis[N];
int level[N];
void bfs(int source){
  queue<int> q;
  q.push(source);
  vis[source] = 1;
  while(!q.empty()){
    int curr_v = q.front();
    q.pop();
    for(auto child: gp[curr_v]){
      if(!vis[child]){
        q.push(child);
        vis[child] = 1;
        level[child] = level[curr_v] + 1;
      }
    }
  }
}
int32_t main (){
  fast;
  int n;
  cin >> n;
  for(int i=0 ; i<n-1 ; ++i){
    int v1, v2;
    cin >> v1 >> v2;
    gp[v1].push_back(v2);
    gp[v2].push_back(v1);
  }
  bfs(1);
  for(int i=1 ; i<=n ; ++i){
    cout << i << ": " << level[i] << endl;
  }
  return 0;
}
```

0-1 BFS (01_bfs.cpp)

Note: relaxation step inside the loop is empty in the source repo; incomplete.

```
const int N = 1e5+10+10;
vector<pair<int, int>> gp[N];
vector<int> lev(N, 1e9+7);
int n, m;
int bfs(){
  deque<int> q;
  q.push_back(1);
  lev[1] = 0;
  while(!q.empty()){
    int cur_v = q.front();
    q.pop_front();
    for(auto child: gp[cur_v]){
      int child_v = child.first;
      int wt = child.second;
    }
  }
}
int32_t main (){
  fast;
  cin >> n >> m;
  for(int i=0 ; i<m ; ++i){
    int x, y;
    cin >> x >> y;
    gp[x].push_back({y, 0});
    gp[y].push_back({x, 1});
  }
  cout << bfs() << endl;
  return 0;
}
```

Multi-source BFS on grid, 8 directions (multi_source_bfs.cpp)

Note: source-scan loop uses j<n; should be j<m for non-square grids.

```
int n, m;
const int N = 1e3 + 10;
int val[N][N];
int vis[N][N];
int lev[N][N];
vector<pair<int, int>> moverments = {
  {1, 0}, {-1, 0}, {0, 1}, {0, -1},
  {-1, -1}, {-1, 1}, {1, -1}, {1, 1}
};
bool isValid(int x, int y){
  return x>=0 && y>=0 && x<n && y<m;
}
int bfs(int source){
  queue<pair<int, int>> q;
  for(int i=0 ; i<n ; ++i){
    for(int j=0 ; j<n ; ++j){
      if(source == val[i][j]){
        q.push({i, j});
        lev[i][j] = 0;
        vis[i][j] = 1;
      }
    }
  }
  int ans = 0;
  while(!q.empty()){
    auto v = q.front();
    auto v_x = v.first;
    auto v_y = v.second;
    q.pop();
    for(auto move: moverments){
      int childX = v_x + move.first;
      int childY = v_y + move.second;
      if(!isValid(childX, childY))    continue;
      if(vis[childX][childY]) continue;
      q.push({childX, childY});
      lev[childX][childY] = lev[v_x][v_y] + 1;
      vis[childX][childY] = 1;
      ans = max(ans, lev[childX][childY]);
    }
  }
  return ans;
}
void reset(){
  for(int i=0 ; i<n ; ++i){
    for(int j=0 ; j<m ; ++j){
      vis[i][j] = 0;
      lev[i][j] = 0;
    }
  }
}
int32_t main (){
  fast;
```

```
  int t;
  cin >> t;
  while(t--){
    reset();
    cin >> n >> m;
    int mx_val = 0;
    for(int i=0 ; i<n ; ++i){
      for(int j=0 ; j<m ; ++j){
        cin >> val[i][j];
        if(val[i][j] > mx_val){
          mx_val = val[i][j];
        }
      }
    }
    cout << bfs(mx_val) << endl;
  }
  return 0;
}
```

Knight moves BFS on 8x8 board (nakanj.cpp)

```
const int N = 100;
vector<int> gp[N];
int vis[N][N];
int level[N][N];
int getX(string s){
  return s[0] - 'a';
}
int getY(string s){
  return s[1] - '1';
}
vector<pair<int, int>> positions = {
  {-2, 1},
  {-1, 2},
  {1, 2},
  {2, 1},
  {-2, -1},
  {-1, -2},
  {2, -1},
  {1, -2},
};
bool isValid(int x, int y){
  return x>=0 && x<8 && y>=0 && y<8;
}
int bfs(string source, string dest){
  int sourceX = getX(source);
  int sourceY = getY(source);
  int destX = getX(dest);
  int destY = getY(dest);
  queue<pair<int, int>> q;
  q.push({sourceX, sourceY});
  vis[sourceX][sourceY] = 1;
  while(!q.empty()){
    pair<int, int> v = q.front();
    int x = v.first, y = v.second;
    q.pop();
    for(auto movement: positions){
      int childX = movement.first + x;
      int childY = movement.second + y;
      if(!isValid(childX, childY))    continue;
      if(!vis[childX][childY]){
        q.push({childX, childY});
        level[childX][childY] = level[x][y] + 1;
        vis[childX][childY] = 1;
      }
    }
  }
  return level[destX][destY];
}
int32_t main (){
  fast;
  int t;
  cin >> t;
  while(t--){
    memset(vis, 0, sizeof(vis));
    memset(level, 0, sizeof(level));
    string s1, s2;
    cin >> s1 >> s2;
    cout << bfs(s1, s2) << endl;
  }
  return 0;
}
```

Connected components with DFS (connected_components.cpp)

```
const int N = 1e5+10;
vector<bool> vis(N);
vector<int> g[N];
void dfs(int vertex){
  vis[vertex] = true;
  for(auto child: g[vertex]){
    if(vis[child])  continue;
    dfs(child);
  }
}
void solve(){
  int n, e;
  cin >> n >> e;
  for(int i=0 ; i<e ; ++i){
    int a, b;
    cin >> a >> b;
    g[a].push_back(b);
    g[b].push_back(a);
  }
  int ct = 0;
  for(int i=1 ; i<=n ; ++i){
    if(vis[i])  continue;
    dfs(i);
    ct++;
  }
  cout << ct << endl;
}
int32_t main (){
  fast;
  int t=1;
  // cin>>t;
  while(t--)
    solve();
  return 0;
}
```

Cycle detection, undirected (cycle.cpp)

```
// #define int long long
const int N = 1e5+10;
vector<bool> vis(N);
vector<int> g[N];
bool isloop = false;
void dfs(int vertex, int parent){
  vis[vertex] = true;
  for(auto child: g[vertex]){
    if(child == parent) continue;
    if(vis[child]){
      isloop = true;
      return;
    }
    dfs(child, vertex);
  }
}
bool isCycle(int V, vector<vector<int>>& edges) {
  int n = V, e = edges.size();
  for(int i=0 ; i<e ; ++i){
    g[edges[i][0]].push_back(edges[i][1]);
    g[edges[i][1]].push_back(edges[i][0]);
  }
  for(int i=0 ; i<n ; ++i){
    if(!vis[i]){
      dfs(i, -1);
      if(isloop)  return true;
    }
  }
  return false;
  // for(int i=0 ; i<n ; i++){
  //     for(int j=0 ; j<g[i].size() ; j++){
  //         cout << g[i][j] << " ";
  //     }
  //     cout << endl;
  // }
}
int32_t main (){
  fast;
  int t=1;
  // cin>>t;
  int v = 4;
  vector<vector<int>> edges = {{0,1}, {1,2}, {2,3}};
  while(t--)
    cout << isCycle(v, edges);
  return 0;
}
```

Height and depth of tree nodes (height_depth.cpp)

```
const int N = 1e5+10;
vector<int> gp[N];
vector<int> height(N, 0);
vector<int> depth(N, 0);
void dfs(int vertex, int par = 0){
  for(auto child: gp[vertex]){
    if(child == par) continue;
    depth[child] = depth[vertex] + 1;
    dfs(child, vertex);
    height[vertex] = max(height[vertex], height[child]+1);
  }
}
int32_t main (){
  fast;
  int n;
  cin >> n;
  for(int i=0 ; i<n-1 ; ++i){
    int v1, v2;
    cin >> v1 >> v2;
    gp[v1].push_back(v2);
    gp[v2].push_back(v1);
  }
  dfs(1);
  cout << "height, depth" << endl;
  for(int i=1 ; i<=n ; ++i){
    cout << height[i] << ", " << depth[i] << endl;
  }
  return 0;
}
```

Subtree sum / count (subtree_sum.cpp)

```
const int N = 1e5+10;
vector<int> gp[N];
vector<int> subtree_sum(N);
vector<int> even_ct(N);
void dfs(int vertex, int par=0){
  subtree_sum[vertex] += vertex;
  if(vertex%2 == 0)   even_ct[vertex]++;
  for(auto child: gp[vertex]){
    if(child == par) continue;
    dfs(child, vertex);
    subtree_sum[vertex] += subtree_sum[child];
    even_ct[vertex] += even_ct[child];
  }
}
int32_t main (){
  fast;
  int n;
  cin >> n;
  for(int i=0 ; i<n-1 ; ++i){
    int a, b;
    cin >> a >> b;
    gp[a].push_back(b);
    gp[b].push_back(a);
  }
  dfs(1);
  for(int i=1 ; i<=n ; ++i){
    cout << subtree_sum[i] << " " << even_ct[i] << endl;
  }
  return 0;
}
```

Edge deletion, max product of two parts (edge_deletion.cpp)

```
const int N = 1e5+10;
vector<int> gp[N];
vector<int> subtree_sum(N);
vector<int> val = {11, 12};
void dfs(int vertex, int par = -1){
  subtree_sum[vertex] = val[vertex-1];
  for(auto child: gp[vertex]){
    if(child == par)    continue;
    dfs(child, vertex);
    subtree_sum[vertex] += subtree_sum[child];
  }
}
int32_t main (){
  fast;
  int n;
  cin >> n;
  for(int i=0 ; i<n-1 ; ++i){
    int v1, v2;
    cin >> v1 >> v2;
    gp[v1].push_back(v2);
    gp[v2].push_back(v1);
  }
  dfs(1);
  int maxprod = 1;
  const int M = 1e9+7;
  for(int i=2 ; i<=n ; ++i){
    int part1 = subtree_sum[i];
    int part2 = subtree_sum[1] - part1;
    maxprod = max(maxprod, (part1*part2)%M);
  }
  cout << maxprod << endl;
  return 0;
}
```

Tree diameter, two DFS (diameter.cpp)

```
const int N = 1e5+10;
vector<int> gp[N];
vector<int> depth(N);
void dfs(int vertex, int par = -1){
  for(auto child: gp[vertex]){
    if(child == par)    continue;
    depth[child] = depth[vertex] + 1;
    dfs(child, vertex);
  }
}
int32_t main (){
  fast;
  int n;
  cin >> n;
  for(int i=0 ; i<n-1 ; ++i){
    int v1, v2;
    cin >> v1 >> v2;
    gp[v1].push_back(v2);
    gp[v2].push_back(v1);
  }
  dfs(1);
  int max_depth = -1, mx_node;
  for(int i=1 ; i<=n ; ++i){
    if(max_depth < depth[i]){
      max_depth = depth[i];
      mx_node = i;
    }
  }
  depth.assign(N, 0);
  dfs(mx_node);
  max_depth = -1;
  for(int i=1 ; i<=n ; ++i){
    if(max_depth < depth[i]){
      max_depth = depth[i];
    }
  }
  cout << max_depth << endl;
  return 0;
}
```

LCA via root paths (lca.cpp)

```
const int N = 1e5+10;
vector<int> gp[N];
vector<int> parent(N);
void dfs(int vertex, int par = -1){
  parent[vertex] = par;
  for(auto child: gp[vertex]){
    if(child == par)    continue;
    dfs(child, vertex);
  }
}
vector<int> path(int node){
  vector<int> ans;
  while(node!=-1){
    ans.push_back(node);
    node = parent[node];
  }
  reverse(all(ans));
  return ans;
}
int32_t main (){
  fast;
  int n;
  cin >> n;
  for(int i=0 ; i<n-1 ; ++i){
    int v1, v2;
    cin >> v1 >> v2;
    gp[v1].push_back(v2);
    gp[v2].push_back(v1);
  }
  dfs(1);
  vector<int> path_a = path(9);
  vector<int> path_b = path(11);
  // for(auto it: path_a){
  //     cout << it << " ";
  // }
  // cout << "\n";
  // for(auto it: path_b){
  //     cout << it << " ";
  // }
  // cout << "\n";
  int lca = -1;
  for(int i=0 ; i<min(path_a.size(), path_b.size()) ; ++i){
    if(path_a[i] == path_b[i]){
      lca = path_a[i];
    }else{
      break;
    }
  }
  cout << lca << endl;
  return 0;
}
```

Dijkstra with set (dijkstra.cpp)

```
const int INF = INT_MAX;
const int N = 1e3 + 10;
int dijkstra(int source, int n, vector<pair<int, int>> g[]){
  vector<int> vis(N, 0);
  vector<int> dist(N, INF);
  set<pair<int, int>> st;
  dist[source] = 0;
  st.insert({0, source});
  while(!st.empty()){
    auto v = *st.begin();
    int dist_v = v.first;
    int node_v = v.second;
    st.erase(st.begin());
    if(vis[node_v] == 1) continue;
    vis[node_v] = 1;
    for(auto it: g[node_v]){
      int node = it.first;
      int wt = it.second;
      if(wt + dist_v < dist[node]){
        dist[node] = wt+dist_v;
        st.insert({dist[node], node});
      }
    }
  }
  for(int i=1 ; i<=n ; ++i){
    if(dist[i] == INF){
      return -1;
    }
  }
  return *max_element(dist.begin()+1, dist.begin()+n+1);
}
int networkDelayTime(vector<vector<int>>& times, int n, int k) {
  vector<pair<int, int>> g[N];
  for(auto it: times){
    g[it[0]].push_back({it[1], it[2]});
  }
  return dijkstra(k, n, g);
}
int32_t main (){
  fast;
  vector<vector<int>> times = {{2,1,1}, {2,3,1}, {3,4,1}};
  int n=4, k=2;
  cout << networkDelayTime(times, n, k);
  return 0;
}
```

Floyd-Warshall (floyd_warshall.cpp)

```
// O(n^3)
const int N = 510;
const int INF = INT_MAX;
int dist[N][N];
int32_t main (){
  fast;
  for(int i=0 ; i<N ; ++i){
    for(int j=0 ; j<N ; ++j){
      if(i == j)  dist[i][j] = 0;
      else dist[i][j] = INF;
    }
  }
  int n, m;   cin >> n >> m;
  for(int i=0 ; i<m ; ++i){
    int x, y, wt;   cin >> x >> y >> wt;
    dist[x][y] = wt;    // For directed graph
  }
  for(int k=1 ; k<=n ; ++k){
    for(int i=1 ; i<=n ; ++i){
      for(int j=1 ; j<=n ; ++j){
        if(dist[i][j] != INF && dist[k][j] != INF){     // This if
          condition for handling negative weights
          dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
        }
      }
    }
  }
  for(int i=1 ; i<=n ; ++i){
    for(int j=1 ; j<=n ; ++j){
      if(dist[i][j] != INF){
        cout << dist[i][j] << ' ';
      }else{
        cout << "INF" << ' ';
      }
    }
    cout << endl;
  }
  return 0;
}
```

CF Greg and Graph: reverse-order Floyd-Warshall (Greg and Graph_cf.cpp)

```
// O(n^3)
const int N = 510;
const int INF = INT_MAX;
int32_t main (){
  fast;
  int n;  cin >> n;
  vector<vector<int>> dist(n+1, vector<int> (n+1));
  for(int i=1 ; i<=n ; ++i){
    for(int j=1 ; j<=n ; ++j){
      cin >> dist[i][j];
    }
```

```
  }
  vector<int> x(n);
  for(auto &it: x)    cin >> it;
  reverse(all(x));
  vector<int> ans;
  for(int k=0 ; k<n ; ++k){
    for(int i=1 ; i<=n ; ++i){
      for(int j=1 ; j<=n ; ++j){
        dist[i][j] = min(dist[i][j], dist[i][x[k]] + dist[x[k]][j]);
      }
    }
    int sum = 0;
    for(int i=0 ; i<=k ; ++i){
      for(int j=0 ; j<=k ; ++j){
        sum += dist[x[i]][x[j]];
      }
    }
    ans.push_back(sum);
  }
  reverse(all(ans));
  for(auto it: ans){
    cout << it << ' ';
  }
  return 0;
}
```

Tarjan SCC + condensation DAG (Tarjan.cpp)

```
struct TarjanSCC{
  int n;
  int timer=0;
  int scc_count=0;
  vector<vector<int>>g;
  vector<vector<int>>dag;
  vector<int>st;
  vector<bool>in_st;
  vector<int>tin; // Entry time
  vector<int>low; // Lowest reachable discovery time
  vector<int>scc_id;
  set<pair<int,int>>edges;
  TarjanSCC(int n,const
    vector<vector<int>>&g):g(g),n(n),in_st(n+1),tin(n+1,-1),low(n+1,-1),
      scc_id(n+1){
    st.reserve(n);
    for(int i=1;i<=n;++i){
      if(tin[i]==-1){
        dfs(i);
      }
    }
    build_dag();
  }
  void dfs(int v){
    st.push_back(v);
    in_st[v]=true;
    tin[v]=low[v]=timer++;
    for(auto u:g[v]){
      if(tin[u]==-1){
        dfs(u);
        low[v]=min(low[v],low[u]);
      }else if(in_st[u]){
        low[v]=min(low[v],tin[u]);
      }
    }
    if(low[v]==tin[v]){
      while(true){
        int u=st.back();
        st.pop_back();
        in_st[u]=false;
        scc_id[u]=scc_count;
        if(u==v)    break;
      }
      scc_count++;
    }
  }
  void build_dag(){
    dag.assign(scc_count,{});
    for(int u=1;u<=n;++u){
      for(auto v:g[u]){
        if(scc_id[u]!=scc_id[v]){
          edges.insert({scc_id[u],scc_id[v]});
        }
      }
    }
    for(auto [u,v]:edges){
      dag[u].push_back(v);
    }
  }
};
int main(){
  int n,m;cin>>n>>m;
  vector<vector<int>>g(n+1);
  for(int i=0;i<m;++i){
    int u,v;cin>>u>>v;
    g[u].push_back(v);
  }
  TarjanSCC ts(n,g);
  cout<<ts.scc_count<<"\n";
  for(int i=1;i<=n;++i){
    cout<<ts.scc_id[i]<<" ";
  }
  cout<<"\n";
  for(int i=0;i<ts.scc_count;++i){
    cout<<i<<": ";
    for(auto v:ts.dag[i]){
      cout<<v<<" ";
    }
    cout<<"\n";
  }
  return 0;
}
```

Kosaraju SCC + condensation DAG (kosaraju.cpp)

```
vvi g, rg, dag;
vi vis, comp;
stack<int> st;
void dfs1(int u){
  vis[u] = 1;
  for(int v : g[u]){
    if(!vis[v])
      dfs1(v);
  }
  st.push(u);
}
void dfs2(int u, int scc){
  vis[u] = 1;
  comp[u] = scc;
  for(int v : rg[u]){
    if(!vis[v])
      dfs2(v, scc);
  }
}
signed main(){
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int n, m;
  cin >> n >> m;
  g.assign(n + 1, {});
  rg.assign(n + 1, {});
  for(int i = 0; i < m; i++){
    int u, v;
    cin >> u >> v;
    g[u].push_back(v);
    rg[v].push_back(u);
  }
  // ---------- First DFS ----------
  vis.assign(n + 1, 0);
  for(int i = 1; i <= n; i++)
    if(!vis[i])
      dfs1(i);
  // ---------- Second DFS ----------
  vis.assign(n + 1, 0);
  comp.assign(n + 1, 0);
  int scc = 0;
  while(!st.empty()){
    int u = st.top();
    st.pop();
    if(!vis[u]){
      ++scc;
      dfs2(u, scc);
    }
  }
  // ---------- Condensation DAG ----------
  dag.assign(scc + 1, {});
  for(int u = 1; u <= n; u++){
    for(int v : g[u]){
      if(comp[u] != comp[v])
        dag[comp[u]].push_back(comp[v]);
    }
  }
  // Remove duplicate edges
  for(int i = 1; i <= scc; i++){
    sort(dag[i].begin(), dag[i].end());
    dag[i].erase(unique(dag[i].begin(), dag[i].end()), dag[i].end());
  }
  // ---------- Output ----------
  cout << "Number of SCC = " << scc << "\n";
  cout << "Component of each node:\n";
  for(int i = 1; i <= n; i++)
    cout << comp[i] << " ";
  cout << "\n";
  return 0;
}
```

Topological sort, Kahn (kahn's algo (topo sort).cpp)

```
/*
Kahn's Algorithm (Topological Sort)
Works for Directed Graph
Returns:
- Topological ordering if graph is DAG.
- {} if cycle exists.
*/
vector<int> topo_sort(int n, vector<vector<int>> &g){
  vector<int> indegree(n + 1, 0);
  // Calculate indegrees
  for(int u = 1; u <= n; ++u){
    for(int v : g[u])
    indegree[v]++;
  }
  // Agar lexicographically smallest topological order chahiye
  // priority_queue<int, vector<int>, greater<int>> q;
  queue<int> q;
  // Push all nodes with indegree 0
  for(int i = 1; i <= n; ++i){
    if(indegree[i] == 0)
      q.push(i);
  }
  vector<int> topo;
  while(!q.empty()){
    int u = q.front();
    q.pop();
    topo.push_back(u);
    for(int v : g[u]){
      indegree[v]--;
      if(indegree[v] == 0)
        q.push(v);
    }
  }
  // Cycle exists
  if((int)topo.size() != n)
    return {};
  return topo;
}
```

Topological sort + cycle check, DFS (topo sort.cpp)

```
/*
Topo Sort + Cycle Detection (DFS)
Works for Directed Graph
If Cycle exists => Not DAG
*/
// Note: If below function returns {} then provided graph is not DAG.
vi topo_sort(int n,vvi &g){
```

```
  vi vis(n+1),topo;
  bool ok=false;
  auto dfs=[&](int u,auto&&self)->void{
    for(auto v:g[u]){
      if(vis[v]==0){
        vis[v]=1;
        self(v,self);
      }else if(vis[v]==1){
        ok=true;
        return;
      }
      if(ok) return;
    }
    vis[u]=2;
    topo.pb(u);
  };
  for(int i=1;i<=n;++i){
    if(vis[i]==0){
      vis[i]=1;
      dfs(i,dfs);
      if(ok)
        return {};
    }
  }
  reverse(all(topo));
  return topo;
}
```

5. Dynamic Programming

Digit DP (digitDP.cpp)

```
string num;
// dp[pos][tight][started][state]
int dp[20][2][2][170];
// -----------------------------------------------------
// state = modify according to problem
// -----------------------------------------------------
int dfs(int pos, bool tight, bool started, int state){
  // Base Case
  if(pos==num.size()){
    // Check validity here
    return 1;
  }
  if(dp[pos][tight][started][state]!=-1)
    return dp[pos][tight][started][state];
  int ans=0;
  int limit=tight?(num[pos]-'0'):9;
  for(int digit=0;digit<=limit;++digit){
    bool nextTight=tight&&(digit==limit);
    bool nextStarted=started||(digit!=0);
    int nextState=state;
    if(nextStarted){
      // Update state
      // nextState=...
    }
    ans+=dfs(
      pos+1,
      nextTight,
      nextStarted,
      nextState
    );
  }
  return dp[pos][tight][started][state]=ans;
}
int solve(int x){
  if(x<0)
    return 0;
  num=to_string(x);
  memset(dp,-1,sizeof(dp));
  return dfs(0,1,0,0);
}
signed main(){
  int L,R;
  cin>>L>>R;
  cout<<solve(R)-solve(L-1);
}
```

TSP bitmask DP, recursive (tsp.cpp)

```
int n;
int dist[20][20];
int visited_all;
int dp[1048576][20];
int tsp(int mask,int pos){
  // Base Case
  if(mask==visited_all){
    return dist[pos][0];
  }
  if(dp[mask][pos]!=-1)   return dp[mask][pos];
  int ans=INT_MAX;
  for(int city=0;city<n;++city){
    if(!((1LL<<city)&mask)){
      int newAns=dist[pos][city]+tsp(mask|(1LL<<city),city);
      ans=min(ans,newAns);
    }
  }
  return dp[mask][pos]=ans;
}
void solve(){
  cin>>n;
  memset(dp,-1,sizeof(dp));
  for(int i=0;i<n;++i)
    for(int j=0;j<n;++j)
      cin>>dist[i][j];
  visited_all=(1LL<<n)-1;
  cout<<tsp(1,0);
}
```

TSP bitmask DP, iterative (tsp_iterative.cpp)

```
const int INF=4e18;
signed main(){
  ios::sync_with_stdio(false);
  cin.tie(nullptr);
  int n;cin>>n;
  vvi dist(n,vi(n));
  for(int i=0;i<n;++i)
    for(int j=0;j<n;++j)
      cin>>dist[i][j];
  vvi dp(1LL<<n,vi(n,INF));
  dp[1][0]=0;
  for(int mask=1;mask<(1LL<<n);++mask){
    if(!(mask&1)) continue;
    for(int last=0;last<n;++last){
      if(!(mask&(1LL<<last))) continue;
      if(dp[mask][last]==INF) continue;
      for(int nxt=0;nxt<n;++nxt){
        if(mask&(1LL<<nxt)) continue;
        dp[mask|(1LL<<nxt)][nxt]=min(
          dp[mask|(1LL<<nxt)][nxt],
          dp[mask][last]+dist[last][nxt]
        );
      }
    }
  }
  int ans=INF;
  for(int last=1;last<n;++last)
    ans=min(ans,dp[(1LL<<n)-1][last]+dist[last][0]);
  cout<<ans;
}
```

6. Extra

Codeforces problem filter by tag and rating (cf problem filter, python)

```
import requests
data = requests.get(
  "https://codeforces.com/api/problemset.problems"
).json()
problems = data["result"]["problems"]
questions = []
for p in problems:
  rating = p.get("rating")
  if p["tags"] == ["dp"] and rating and 800 <= rating <= 2000:
    link = f"https://codeforces.com/problemset/problem/{p['contestId']}/
      {p['index']}"
    questions.append((
      rating,
      p["name"],
      link
    ))
questions.sort()
for rating, name, link in questions:
  print(f"{rating} | {name}")
  print(link)
```